

```

    else:
        print("Epoch {0} complete".format(j))

def update_mini_batch(self, mini_batch, eta):
    """Update the network's weights and biases by applying
    gradient descent using backpropagation to a single mini batch.
    The ``mini_batch`` is a list of tuples ``(x, y)``, and ``eta``
    is the learning rate."""
    nabla_b = [np.zeros(b.shape) for b in self.biases]
    nabla_w = [np.zeros(w.shape) for w in self.weights]
    for x, y in mini_batch:
        delta_nabla_b, delta_nabla_w = self.backprop(x, y)
        nabla_b = [nb+dnb for nb, dnb in zip(nabla_b, delta_nabla_b)]
        nabla_w = [nw+dnw for nw, dnw in zip(nabla_w, delta_nabla_w)]
    self.weights = [w-(eta/len(mini_batch))*nw
                     for w, nw in zip(self.weights, nabla_w)]
    self.biases = [b-(eta/len(mini_batch))*nb
                    for b, nb in zip(self.biases, nabla_b)]

def backprop(self, x, y):
    """Return a tuple ``(nabla_b, nabla_w)`` representing the
    gradient for the cost function C_x. ``nabla_b`` and

```

```

nabla_w = [np.zeros(w.shape) for w in self.weights]
# feedforward
activation = x
activations = [x] # list to store all the activations, layer by layer
zs = [] # list to store all the z vectors, layer by layer
for b, w in zip(self.biases, self.weights):
    z = np.dot(w, activation)+b
    zs.append(z)
    activation = sigmoid(z)
    activations.append(activation)
# backward pass
delta = self.cost_derivative(activations[-1], y) * \
    sigmoid_prime(zs[-1])
nabla_b[-1] = delta
nabla_w[-1] = np.dot(delta, activations[-2].transpose())
# Note that the variable l in the loop below is used a little
# differently to the notation in Chapter 2 of the book. Here,
# l = 1 means the last layer of neurons, l = 2 is the
# second-last layer, and so on. It's a renumbering of the
# scheme in the book, used here to take advantage of the fact
# that Python can use negative indices in lists.
for l in range(2, self.num_layers):
    z = zs[-l]

```

Sigmoid $\rightarrow \delta^L = \nabla C \cdot \sigma'(z^L)$ } Función de costo
 = self.cost_derivative

- Programadores de Linux { Proceso realizar poca cantidad de código, no tan mucho. Dependiendo de la empresa:

$$(x_1, x_2, x_3) \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

$$\frac{\partial C_x}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l ;$$

$$\frac{\partial C_x}{\partial b_j^l} = \delta_j^l$$

Ejemplo de operación

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}_{3 \times 1} \begin{pmatrix} x_1, x_2, x_3 \end{pmatrix}_{1 \times 3} = \begin{pmatrix} x_1 y_1 & x_2 y_1 & x_3 y_1 \\ y_2 x_1 & y_2 x_2 & y_2 x_3 \\ y_3 x_1 & y_3 x_2 & y_3 x_3 \end{pmatrix}$$

```

123
124 def evaluate(self, test_data):
125     """Return the number of test inputs for which the neural
126     network outputs the correct result. Note that the neural
127     network's output is assumed to be the index of whichever
128     neuron in the final layer has the highest activation."""
129     test_results = [(np.argmax(self.feedforward(x)), y)
130                     for (x, y) in test_data]
131     return sum(int(x == y) for (x, y) in test_results)
132
133 def cost_derivative(self, output_activations, y):
134     """Return the vector of partial derivatives \partial C_x /
135     \partial a for the output activations."""
136     return (output_activations - y)
137
138 ##### Miscellaneous functions
139 def sigmoid(z):
140     """The sigmoid function."""
141     return 1.0 / (1.0 + np.exp(-z))
142
143 def sigmoid_prime(z):
144     """Derivative of the sigmoid function."""
145     return sigmoid(z) * (1 - sigmoid(z))
146

```

• vector de 10 entidades

• neurona de mayor valor.

• Funciones de activación son continuas.

• índice de la neurona más

• ciclamos activada.

• Evalúa la red neuronal para saber que tan eficiente es.

Función de costo de un dato:

$$C_x = \frac{1}{2} (y - x)^2 ;$$

Derivamos

$$\frac{\partial C_x}{\partial x} = (x - y)$$


```
137
138 ##### Miscellaneous functions
139 def sigmoid(z):
140     """The sigmoid function."""
141     return 1.0/(1.0+np.exp(-z))
142
143 def sigmoid_prime(z):
144     """Derivative of the sigmoid function."""
145     return sigmoid(z)*(1-sigmoid(z))
146
```

• Paquetes como
Tensor flow

↳ Construyen grafos para
obtener las derivadas

¿Cómo usamos este código?
→ vs code

• construimos

```
net = network.Network([784, 30, 10])
```

↑
Píxeles

↑
Neuronas
capa
intermedia

Neuronas
de salida

Épocas

Stochastic gradient descent

Entrenamos

```
def SGD(self, training_data, epochs, mini_batch_size, eta,
        test_data = None)
```

• mini-batch
tiene que ser de
tamaño 5.

• learning rate = generalmente es de 0.1

↳ es un hiperparámetro.

En código no es el mismo tratamiento para el caso de las
matrices respecto a lápiz y papel.

Primera tarea trataré sobre modificar este código

En teoría ya vimos
la básica de una
red neuronal con el
código network que
compartió el prof. Jorge.

Resaltadas

ejemplo.py DataVisualization.py network.py

→ Aprender lo básico de git.

